Wokwi setup en VSCode

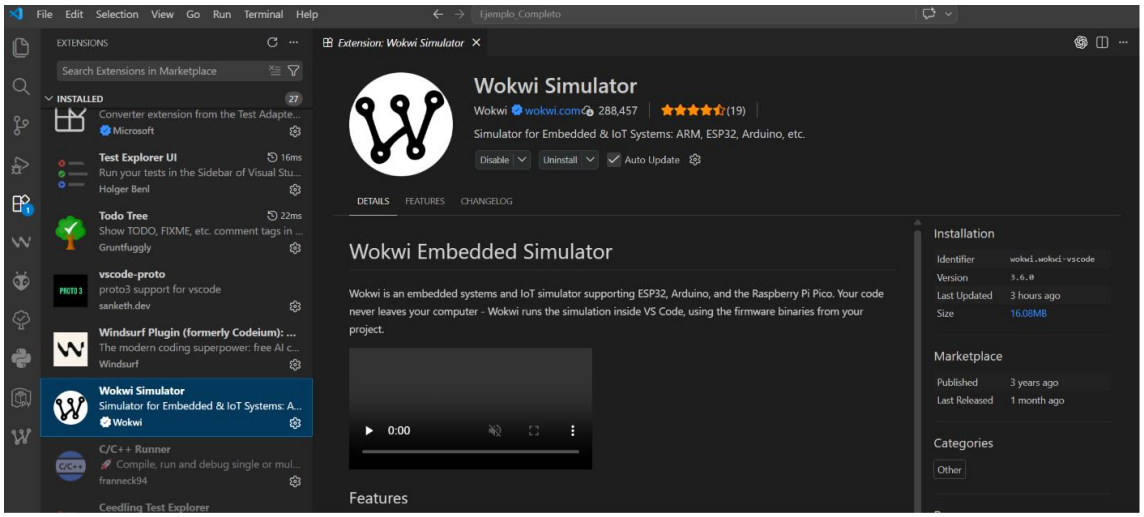
Autor: Mg. Ing. Juan P. Pedroni.

Wokwi es un simulador de sistemas embebidos e IoT que soporta ESP32, Arduino, STM32 y Raspberry Pi Pico, entre otras plataformas. Su extensión para Visual Studio Code permite correr la simulación dentro del IDE, usando el firmware compilado por PlatformIO: el mismo binario que se grabaría en la placa real se ejecuta sobre el modelo simulado del micro. Esto permite probar código y circuitos sin hardware físico.

Requisitos previos: tener instalado Visual Studio Code con la extensión PlatformIO IDE, y una cuenta gratuita en wokwi.com.

Descargar la extensión para VSCode

1. Abrir la vista de extensiones con `Ctrl+Shift+X`.
2. Buscar “Wokwi Simulator” (publicada por Wokwi).
3. Hacer click en Install y esperar a que finalice la instalación.



La extensión no compila el código: solo simula el binario que genera PlatformIO. Por eso siempre hay que compilar el proyecto antes de simular.

Conseguir una licencia gratuita

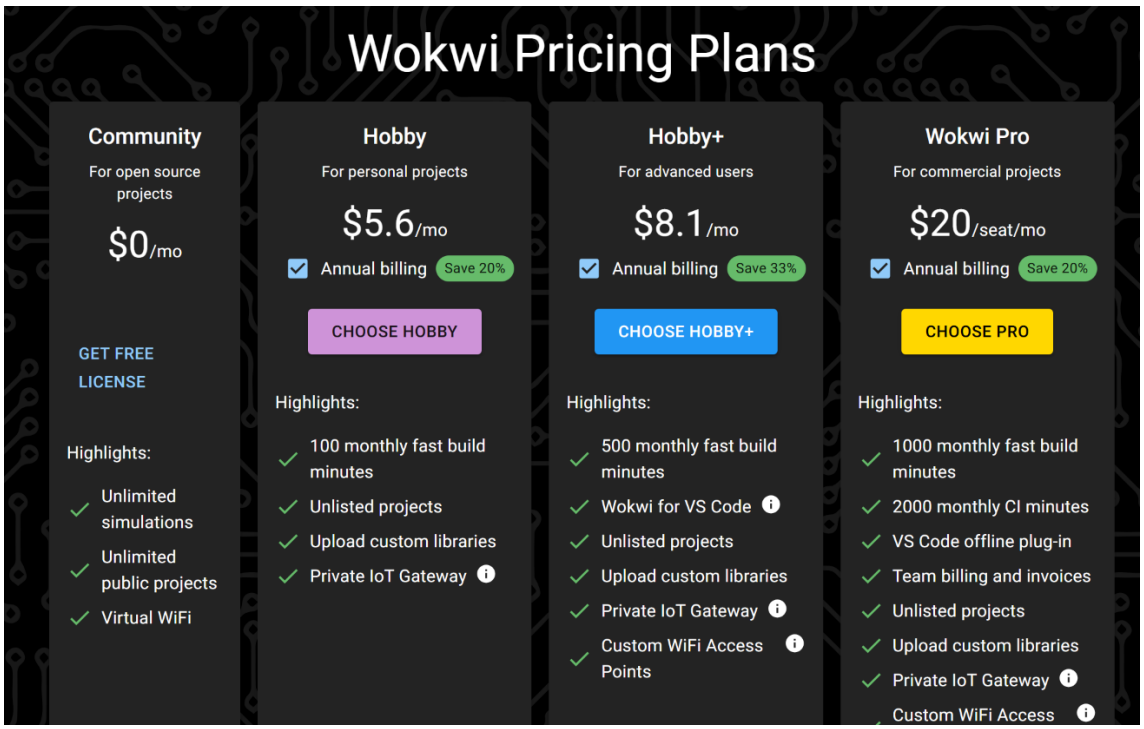
La extensión requiere una licencia para funcionar. El plan Community es gratuito para uso educativo y proyectos open source, tiene una vigencia de 30 días y se renueva repitiendo este mismo procedimiento.

1. Ingresar a wokwi.com e iniciar sesión (o crear una cuenta si todavía no se tiene).

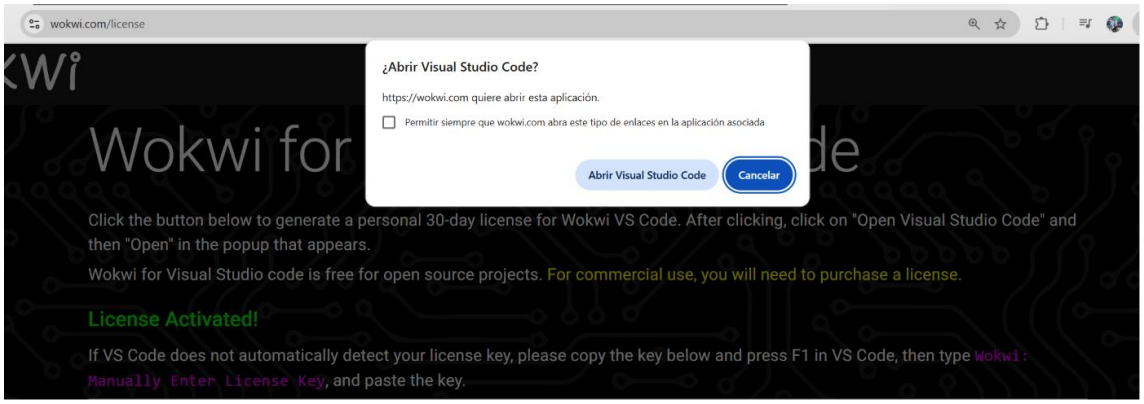
https://wokwi.com



2. Hacer click en Pricing.



3. En el plan Community, hacer click en GET FREE LICENSE. El navegador abre la página wokwi.com/license, genera la licencia personal de 30 días y pide abrir VSCode.



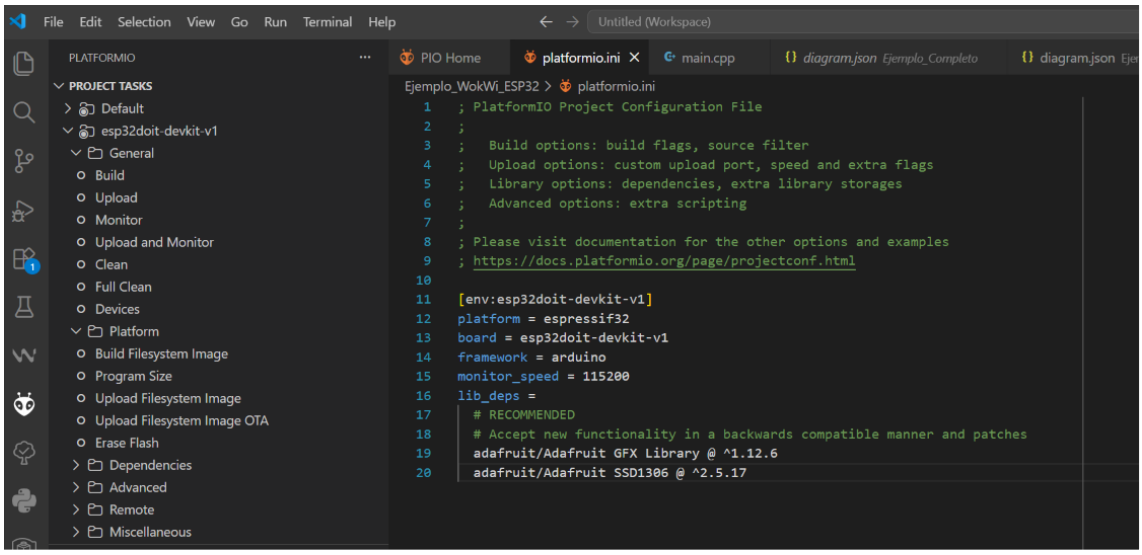
4. Aceptar con “Abrir Visual Studio Code”: la extensión detecta la licencia y la activa automáticamente. Si VSCode no la detecta, copiar la clave que muestra la página, presionar `F1` en VSCode, ejecutar `Wokwi: Manually Enter License Key` y pegarla.

Criterio de validación: la página muestra “License Activated!” y VSCode notifica que la licencia quedó instalada. Cuando venza, cualquier intento de simular va a pedir renovarla: se repite este procedimiento desde Pricing.

Crear un proyecto de PlatformIO

1. Desde PIO Home, hacer click en New Project.
2. Completar: Name (por ejemplo `Ejemplo_WokWi_ESP32`), Board: DOIT ESP32 DEVKIT V1, Framework: Arduino.
3. Editar `platformio.ini` para fijar la velocidad del monitor serie y declarar las librerías del display OLED usado en el ejemplo:

```
[env:esp32doit-devkit-v1]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino
monitor_speed = 115200
lib_deps =
  adafruit/Adafruit GFX Library @ ^1.12.6
  adafruit/Adafruit SSD1306 @ ^2.5.17
```

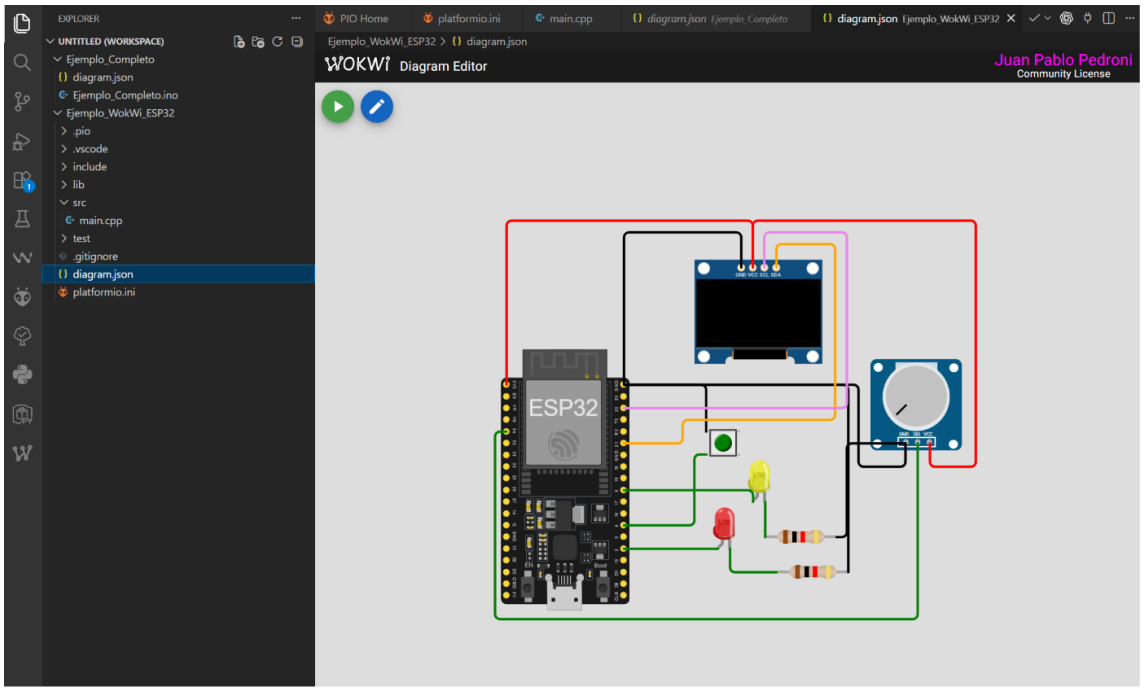


4. Escribir el código de la aplicación en `src/main.cpp` y compilar con Build (el tilde ✓ de la barra inferior o `Ctrl+Alt+B`). El binario queda en `.pio/build/esp32doit-devkit-v1/firmware.bin`, que es el archivo que Wokwi va a simular.

Configurar el simulador: diagram.json y wokwi.toml

Para simular, Wokwi necesita dos archivos en la raíz del proyecto: `diagram.json`, que describe el circuito, y `wokwi.toml`, que le indica dónde está el firmware compilado.

Crear un archivo `diagram.json` en la raíz del proyecto y cargar el contenido del JSON que tiene las conexiones (el del ejemplo está en la carpeta `Ejemplo_WokWi_ESP32`). Al abrir el archivo, VSCode muestra el editor gráfico de Wokwi, donde se puede armar o modificar el circuito agregando componentes y cableándolos con el mouse.

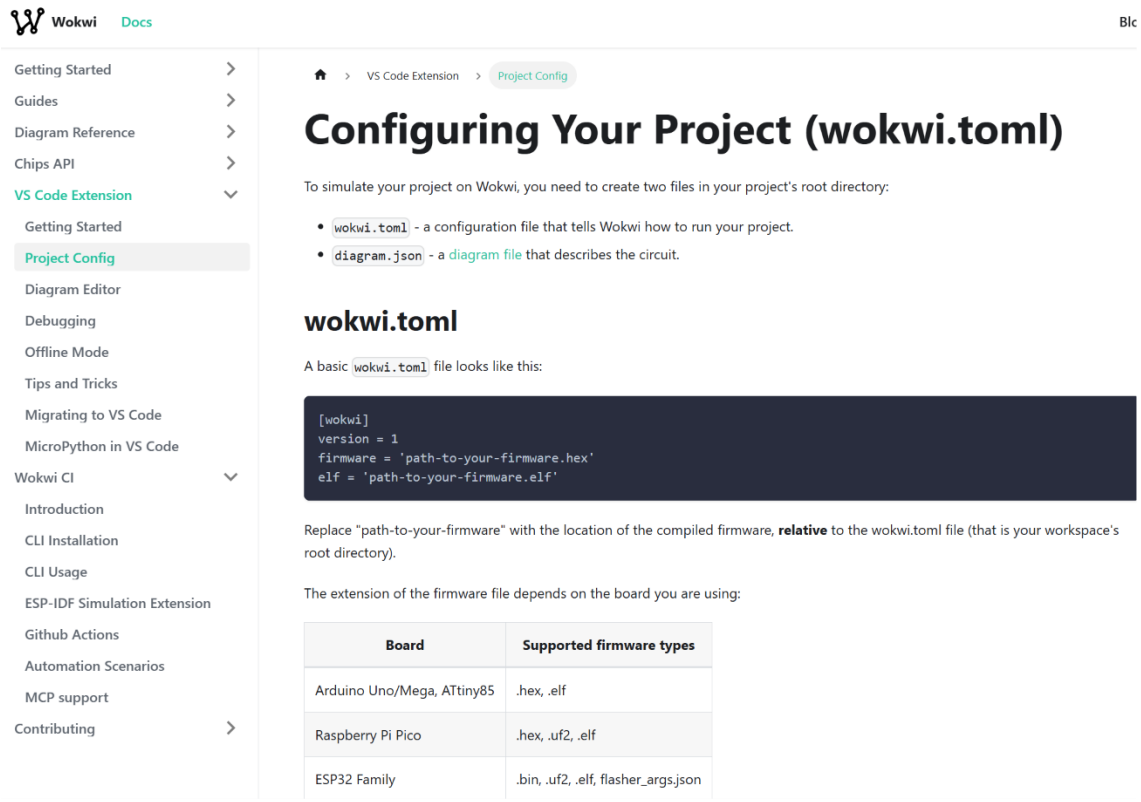


Crear el archivo de configuración `wokwi.toml`, también en la raíz, con las rutas al firmware relativas al propio archivo:

```
[wokwi]
version = 1
firmware = '.pio/build/esp32doit-devkit-v1/firmware.bin'
elf = '.pio/build/esp32doit-devkit-v1/firmware.elf'
```

Si se cambia el nombre del entorno en `platformio.ini`, hay que actualizar estas rutas. La referencia completa del formato está en:

https://docs.wokwi.com/vscode/project-config



Ejecutar la simulación

1. Compilar el proyecto con Build. Wokwi simula el binario ya compilado: después de cada cambio en el código hay que recompilar antes de volver a simular.
2. Presionar `F1` y ejecutar `Wokwi: Start Simulator`, o abrir `diagram.json` y usar el botón de play verde del editor.
3. El monitor serie de la simulación aparece en el panel inferior, con la velocidad configurada en `monitor_speed` (115200 baudios en este ejemplo).

El proyecto de ejemplo: Ejemplo_WokWi_ESP32

El ejemplo integra las entradas y salidas típicas de un trabajo práctico: entrada digital, entrada analógica, salida digital, salida PWM y un periférico I2C.

El circuito

Componente	Conexión al ESP32	Función en el ejemplo
LED rojo + resistencia 1 kΩ	GPIO 2	Parpadeo cada 1 s, no bloqueante
Pulsador	GPIO 4, entrada con pull-up interno (activo en bajo)	Detección de presionado/liberado por sondeo

Potenciómetro	GPIO 34 (canal ADC, pin de entrada solamente)	Lectura analógica de 8 bits (0 a 255)
LED amarillo + resistencia 1 kΩ	GPIO 5 (salida PWM)	Brillo proporcional a la posición del potenciómetro
Display OLED SSD1306 128×64	I2C: SDA en GPIO 21, SCL en GPIO 22, dirección 0x3C	Salida gráfica

El código

El programa completo está en `src/main.cpp`. Los puntos a observar:

- `blink(interval)` parpadea el LED comparando `millis()` contra la última conmutación, sin usar `delay()` largos: el lazo principal queda libre para atender el resto de las tareas.
- `leerEstadoPulsador()` detecta cambios de estado comparando la lectura actual con la anterior (variable `static`). Como el pin usa pull-up interno, presionado se lee como nivel bajo.
- `leerEstadoPote()` lee el ADC configurado en 8 bits con `analogReadResolution(8)`, así el rango 0–255 coincide con el que acepta `analogWrite()`. Solo imprime cuando el valor cambia, para no inundar el monitor serie.
- El valor del potenciómetro se aplica directamente como ciclo de trabajo PWM del LED amarillo con `analogWrite(PIN_LED_PWM, ledPWM)`.
- `displayInit()` inicializa el SSD1306 por I2C en la dirección 0x3C usando las librerías de Adafruit y escribe un texto de prueba.

```
#include <Arduino.h>


#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>


#define PIN_LED 2       // Pin asociado al led
#define PIN_PULS 4      // Pin asociado al pulsador
#define BAUDRATE 115200 // Velocidad de comunicación del puerto serie
#define PIN_POT 34      // Pin asociado al potenciómetro
#define PIN_LED_PWM 5   // Pin asociado al led con regulación de intensidad

// Configuración del display
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
#define OLED_RESET -1    // Reset pin # (or -1 if sharing Arduino reset pin)
#define OLED_ADDRESS 0x3C


Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

```
int leerEstadoPote()
{
  static int poteAnt = 0;
  int pote = analogRead(PIN_POT);
  if (pote != poteAnt)
  {
    Serial.printf("[%lu] Valor del pote: %d%%\n", millis(), map(pote, 0, 255, 0,
100));
    poteAnt = pote;
  }
  return pote;
}

void blink(const unsigned long interval)
{
  static unsigned long previousMillis = 0;
  if (millis() - previousMillis > interval)
  {
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    previousMillis = millis();
  }
}

void leerEstadoPulsador()
{
  static bool estadoPulsadorAnterior = true;
  bool estadoPulsador = digitalRead(PIN_PULS);
  if (estadoPulsador != estadoPulsadorAnterior)
  {
    if (!estadoPulsador)
    {
      Serial.printf("[%lu] Pulsador presionado\n", millis());
    }
    else
    {
      Serial.printf("[%lu] Pulsador liberado\n", millis());
    }
    estadoPulsadorAnterior = estadoPulsador;
  }
}

void displayInit()
{
  if (display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS))
  {
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(5, 5);
    display.setTextColor(SSD1306_WHITE);
    display.print("Hello There");
    display.display();
  }
  else
  {
```

```
    Serial.println("No se pudo inicializar el display");
  }
}

void setup()
{
  Serial.begin(BAUDRATE);
  Serial.println("Hello, ESP32!");
  Serial.printf("Fecha y hora de compilacion: %s,%s\n", __DATE__, __TIME__);
  // configurar el pin 2 como salida.
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, HIGH);
  // configurar el pin 4 como entrada.
  pinMode(PIN_PULS, INPUT_PULLUP);
  // configurar la precisión del ADC
  analogReadResolution(8);
  // configurar el pin 5 como salida
  pinMode(PIN_LED_PWM, OUTPUT);
  analogWrite(PIN_LED_PWM, 120);
  // inicializar el display
  displayInit();
}

void loop()
{
  blink(1000);
  leerEstadoPulsador();
  int ledPWM = leerEstadoPote();
  analogWrite(PIN_LED_PWM, ledPWM);
  delay(10);
}
```

Verificación: qué se debería ver

Con la simulación corriendo, el ejemplo se considera funcionando si se cumple todo lo siguiente:

- El monitor serie muestra “Hello, ESP32!” y la fecha y hora de compilación al arrancar.
- El LED rojo conmuta una vez por segundo.
- Al hacer click en el pulsador, el monitor registra “Pulsador presionado” y “Pulsador liberado” con su marca de tiempo en ms.
- Al girar el potenciómetro cambia el brillo del LED amarillo y el monitor imprime el nuevo valor en porcentaje.
- El display OLED muestra “Hello There”.

Si el simulador arranca pero el programa no corre, verificar que el proyecto esté compilado y que las rutas de `wokwi.toml` apunten al `firmware.bin` correcto. Si el display no muestra nada, revisar las conexiones SDA/SCL en `diagram.json` y la dirección I2C (0x3C).